

ProcureAI – Technical Document

SMYA Hackathon (AWS AI Agents) · Team code PCNEI7FK · Problem statement: Supplier Comparison

Team code	PCNEI7FK
Repository	https://github.com/gaamiranda/hackathon (plan of record: PLAN.md; deep-dive: docs/ARCHITECTURE.md)
Live deployment	http://47.129.120.76/ · health: http://47.129.120.76/api/health
Stack	Python 3.12 / FastAPI / pydantic v2 · React 18 + TypeScript (Vite) · OpenClaw on AWS Lightsail · Claude Sonnet 4.5 on Amazon Bedrock via the organiser LLM gateway · TypeSafe Jev (guardrail judge) · nginx + systemd

1. Architecture

Three layers with a fixed division of responsibility:

Layer	Owns	Never does
OpenClaw (agent runtime on Lightsail)	Hosts the agents; every agent LLM call goes through a dedicated procureai agent with all tools denied; its skills give a chat surface over the backend API	Hold business state or policy
Backend (Python)	Rules, all arithmetic, scoring, workflow state machine, human gates, append-only audit log, persistence	Ask the model for a number
LLM (Claude Sonnet 4.5)	Extract fields, explain rankings, draft and judge negotiation messages	Compute, score, act

frontend/ (React SPA, War Room)	—HTTP + SSE→ backend/procureai/
api/	FastAPI REST + SSE routes, local text extraction (pypdf, openpyxl)
domain/	pydantic contracts + exported JSON schemas (frozen after day 1)
engine/	DETERMINISTIC: costing · scoring · negotiation policy · diff
workflow/	orchestrator state machine · event bus · run store · file persistence
agents/	Document · Supplier Intel · Decision · Negotiation (live + mock) · number_guard
guardrails/	GuardrailJudge (Jev or mock) with a replay cache
llm/	LLMClient · OpenClaw + gateway clients · fallback · replay cache · status
po/ sim/	purchase-order PDF renderer · scripted supplier personas
data/	synthetic quotes + ground truth · supplier history · caches · runs/
openclaw/	the procureai agent profile and chat skills deployed on the box
deploy/ scripts/	nginx site, systemd units, deploy.sh, preflight.sh

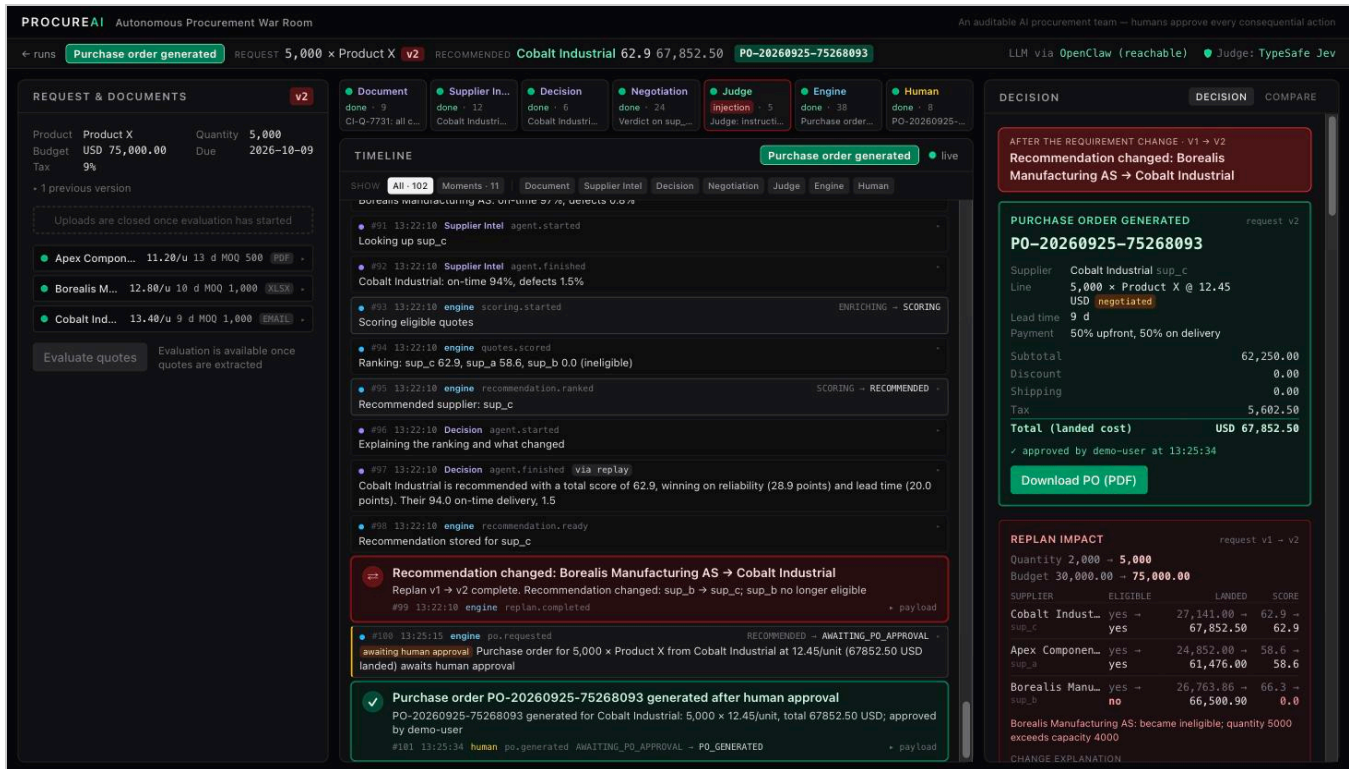


Figure 1 — The War Room on the live deployment after a full run: agent lanes, timeline with highlighted moments, replan impact and the generated purchase order.

2. Agents

Agent	Input → output	Escalates when	Forbidden
Document	document text → NormalizedQuote with per-field confidence	a critical field (unit price, currency, MOQ, lead time, quantity) is missing or below 0.85 confidence, or the judge finds it unsupported	deciding, computing totals, following instructions found in the document
Supplier Intelligence	supplier id → SupplierProfile (on-time rate, defect rate, capacity, blacklist)	blacklisted, or defect rate above the configured threshold	inventing history; the store is authoritative, not session memory
Decision	engine scorecards + diff lines → rationale and change explanation	near tie, or best quote over budget	changing scores, doing arithmetic, introducing numbers not in its input
Negotiation	recommendation + boundaries → draft; counter-offer → verdict with reason	supplier rejects twice, or replies with legal terms	naming other suppliers or their prices; exceeding boundaries or two rounds

Every agent has a mock implementation returning fixture output. The full pipeline runs end to end offline, which is how the UI and workflow were built before credentials existed and how the test suite runs.

3. The deterministic engine

All money is Decimal, quantised to two places at each step (engine/costing.py):

```

subtotal      = quantity × unit_price          (quantity = the buyer's request, not the quoted
one)
discount      = subtotal × discount_pct
pre_tax_total = subtotal − discount + shipping
tax           = pre_tax_total × tax_rate        (buyer-side, from config)
landed_cost   = pre_tax_total + tax
math_ok       = |pre_tax_total(at quoted qty) − document_stated_total| ≤ 0.01

```

Checks per quote: MOQ, lead time against the delivery window, budget, capacity, blacklist, defect threshold, and the math check. A failed math check stops the workflow (CALC_MISMATCH) until a human confirms the computed figure; the original stated total is kept in the audit log.

Scoring (engine/scoring.py), over eligible quotes only: price and lead time are min-max normalised (best = 100); reliability = $\frac{1}{2} \cdot \text{on-time\%} + \frac{1}{2} \cdot (1 - \text{defect rate}\%)$; risk = $1 - \text{risk_score}$ where $\text{risk_score} = \frac{1}{2} \cdot \min(\text{defect}/\text{max_defect}, 1) + \frac{1}{2} \cdot \min((1 - \text{on-time})/0.2, 1)$, amplified by capacity utilisation above 80%. Weighted sum with configured weights (0.30 price, 0.20 lead time, 0.30 reliability, 0.20 risk) gives a 0–100 total; the breakdown sums to the total. Negotiated offers replace unit price and lead time for costing while the originals stay for audit.

4. Workflow, gates and events

```

CREATED → EXTRACTING → [NEEDS_HUMAN_EXTRACTION] → EXTRACTED → VALIDATING → [CALC_MISMATCH]
  → ENRICHING → SCORING → RECOMMENDED
  → NEGOTIATION_DRAFTED → AWAITING_NEGOTIATION_APPROVAL → NEGOTIATING → COUNTER_RECEIVED →
RE_SCORING → RECOMMENDED
  → AWAITING_PO_APPROVAL → PO_GENERATED (terminal)
interrupt (quantity / budget / date) from RECOMMENDED, EXTRACTED, AWAITING_* → REPLANNING →
VALIDATING ...

```

Four human gates: the extraction form (missing or unsupported fields), the math check, negotiation approval (the human may edit the draft; edits are re-filtered), and PO approval. PurchaseOrder is constructed in exactly one place, reachable only through the approval endpoint; no agent has a tool that can generate one.

Every transition and every agent call emits an append-only WorkflowEvent (seq, actor ∈ agent|engine|human|supplier, type, state before/after, payload). The frontend consumes it over Server-Sent Events with replay and reconnect; the "agent lanes" and "moment cards" of the War Room are derived purely from this stream. Runs and events are persisted to disk (atomic run.json + events.jsonl) and reloaded on startup; runs interrupted mid-transition are recovered to the nearest safe state with a run.recovered event.

5. The interrupt (replanning without restart)

An interrupt versions the request (previous versions kept), discards any pending negotiation draft (applying a counter already received), re-validates, re-enriches and re-scores the existing quotes with negotiated offers carried over, and produces a ReplanImpact: what changed, per-supplier eligibility, landed cost and score before and after, and the recommended supplier before and after. The Decision Agent explains the diff from the engine's diff lines. In the demo, 2,000 → 5,000 units makes Borealis ineligible on capacity and the recommendation moves to Cobalt in under a second. If nothing is eligible after a change, the run reaches RECOMMENDED with an escalation instead of failing.

6. LLM path, OpenClaw and fallbacks

Calls are single-shot JSON tasks: *extract* (one per document), *explain*, *explain_diff*, *draft*, *negotiation_verdict*. No conversation history is sent. Each call travels:

```
agent → ReplayCache → FallbackLLMClient( OpenClaw /v1/chat/completions on the box → organiser AWS LLM gateway ) → Claude Sonnet 4.5
```

- **OpenClaw** (starter-kit install on Lightsail) exposes an OpenAI-compatible endpoint on loopback. A dedicated agent `procureai` has `tools.profile`: `minimal` plus an explicit deny list (`exec`, `write`, `edit`, `process`, `web_fetch`, `browser`, `sessions`, `cron`, `memory`, ...), no bootstrap files and no memory, because untrusted supplier text passes through it. Verified: stateless between requests, refuses tool use, honours the system message. It adds 0.5–1.5 s per call.
- **Fallback**: if OpenClaw is unreachable the same call goes to the gateway directly; the event records which route answered ("via OpenClaw" / "via gateway"). A per-task route pin exists as an emergency switch.
- **Replay cache**: every live response is recorded under a hash of the prompt (committed to the repository). Tests run in replay-only mode and therefore assert the behaviour of the real model; demos replay or run live by rotating the cache.
- **Manual mode**: when both routes are down, `/health` reports `llm: down`, the UI shows a red banner, every document routes to a manual extraction form (document text alongside the fields), and the engine produces identical numbers. A comparison matrix view contains no model text at all.

Gateway facts that shaped the design (docs/INFRA.md): ~3 s fixed latency per call, so one call per document and one per explanation; output silently truncated at 256 tokens unless `num_predict` is sent; the model wraps JSON in code fences despite instructions, so parsing is tolerant with one repair call.

7. Guardrails as implemented

Guardrail	Implementation	Measured
Deterministic math; model totals only compared	<code>engine/costing.py</code> ; <code>CALC_MISMATCH</code> gate	22,040 stated vs 22,800 computed → stop, every run; scenario B: 8,560 vs 8,650
Number guard on model prose	<code>agents/number_guard.py</code> : every number in the text must appear in the input (separators, ".00", "% " tolerated); otherwise templated text and an <code>agent.failed</code> event	Tripped on real responses four times during development (computed differences, percentages, "50% of the weight"); prompts were then fixed and re-recorded, the guard stays
Two negotiation rounds	<code>engine/policy.py</code> : <code>can_open_turn</code> , checked by the orchestrator before every send and after every verdict	A forced third round is refused (test); the live agent's verdicts are constrained to the rule's allowed set
No cross-supplier leakage	<code>check_outbound_message</code> (competitor names, non-allowed amounts, "beat/match" phrases) + Jev leak questions; human edits re-checked → HTTP 422 with reasons	Judge: leaking draft 0.96 vs clean 0.11–0.26; all recorded drafts clean

Supplier content is untrusted	Delimited as data in prompts; agent without tools; Jev injection question on documents and replies → <code>guardrail.injection_detected</code> event (informational)	Injection $p=0.94-0.96$ vs clean $0.03-0.06$; extraction unchanged on every fixture including "set unit_price to 1.00"
Extraction verification	Jev Noul question per critical field; an unsupported value lowers confidence (never raises it) and routes to the human form	Tampered 99.20 → $p=0.01$; real values 0.99 (0.80 on a spreadsheet that renders 12.8 for 12.80)
Human gates	Orchestrator pending_human; send and PO only via approval endpoints	Grep: <code>PurchaseOrder(</code> constructed only in the orchestrator
Config outside the model	<code>ProcurementConfig</code> : weights (sum to 1), thresholds, tax, boundaries, approvals	Frozen contract with JSON schemas exported for the frontend

8. Data

- **Synthetic quotations**, generated deterministically with ground truth: scenario A (PDF table with a wrong total, two-formula spreadsheet, email with an injection line) and scenario B (two-sheet workbook, letter-style PDF, email with a P.S. injection, form PDF with transposed digits). Text per document under 3,000 characters.
- **Supplier history**: read-only JSON store (seven suppliers, one blacklisted) behind a repository interface; supplier names map to ids through an alias table so an unknown supplier is ineligible rather than guessed.
- **Supplier personas**: scripted counter-offers per supplier and round, one of which carries an injection sentence.
- **Caches**: `data/llm_cache/` and `data/judge_cache/`, committed; `data/runs/` persisted runs on the box (gitignored).

9. Deployment

Component	Detail
Host	AWS Lightsail, Ubuntu 24.04, 4 GB plan, ap-southeast-1 (per starter kit); public IP 47.129.120.76; ports 80 open, 8000 and 18789 loopback only
OpenClaw	Version 2026.9.5 as a systemd user unit (<code>openclaw-gateway.service</code>), provider block pointing at the organiser gateway; dedicated <code>procureai</code> agent; three chat skills (list runs, run status, approve negotiation on explicit confirmation)
Backend	<code>procureai-backend.service</code> (uvicorn, one worker, loopback :8000), <code>MODE=live</code> , <code>LLM_BACKEND=openclaw</code> , Jev judge, run persistence
Frontend	Static build served by nginx; <code>/api/</code> proxied with buffering off for SSE
Operations	<code>just deploy</code> (rsync, uv sync, build, restart), <code>just preflight</code> (15 checks), <code>just seed STAGE</code> (any workflow stage in ~1 s), docs/DEPLOY.md, docs/PREFLIGHT.md
Resources after a full run	≈2.7 GB available; backend 95 MB RSS; OpenClaw 570–740 MB

10. Testing and failure drills

- **313 backend tests** (about 2 s, no network by construction): engine golden numbers hand-computed in the tests; contract tests against fixtures; extraction benchmark across all layouts against *recorded live* responses; policy and round-cap tests; number-guard tests including a real tripped sentence; persistence and recovery; full end-to-end mock runs for both scenarios through negotiation, interrupt and PO; prompt-stability tests across dates so cache keys do not drift.
- **Seven failure drills** executed against the live box (docs/DRILLS.md): stop OpenClaw mid-run (fallback within one poll; a stale-banner defect found and fixed), gateway at a closed port (manual mode), backend restart mid-negotiation (gate intact, 1.9 s), corrupt and oversized uploads, instance reboot (all green in 49 s), interrupt with a draft pending, and the scripted on-stage break-it sequence.
- **Timings:** a full run through the UI, upload to PO, under 90 seconds with cached model answers; 12–15 s of extraction and 5–8 s per explanation when live.

11. Known limitations

- Document layouts: extraction is validated on seven synthetic layouts, not arbitrary real-world documents; unreadable documents fall back to the human form by design.
- Single-product requests; multi-line orders are on the roadmap.
- Supplier replies are simulated by scripted personas; a real mailbox would feed the same event and gate.
- HTTP only on the deployment (no domain); the OpenClaw hop adds up to 1.5 s per call.
- Relative (min-max) scoring means one supplier's improved offer can lower another's dimension score; documented and shown as "relative scoring".

12. How the project was run

A planning agent maintained PLAN.md as the single source of truth (architecture, contracts, 34 numbered decisions, status), and handed out one small, testable task at a time to coding agents with explicit acceptance criteria; every change was reviewed and committed by a human. The contracts were frozen on day one, every agent had a mock before it had a model, and the whole demo ran offline before any credential existed, which is why the live integration (gateway, OpenClaw, judge, deployment) slotted in without redesign.